

HW05 Task 2 — Phân tích của AI và cuộc săn lỗi diễn giải

Phần **phân tích** là output của AI; phần **rà soát** là của sinh viên. Mọi con số phản bác phải trích được từ `.jtl` thô trong `perf/results/jtl/`.

Ghi chú phạm vi: phần phân tích ở `perf/results/ai-analysis-raw.md` không đến từ một subagent tách biệt như `task-21-brief.md` dự kiến — phiên này bị cấm spawn subagent, và một lần thử gọi tiến trình Claude Code riêng bị chính bộ phân loại của harness chặn. Thay vào đó, output đó là một lượt tính toán trực tiếp, có kỷ luật: chỉ dùng arithmetic tổng hợp thô trên 4 file `.jtl`, không dùng `analyze_jtl.py`, không biết trước ý nghĩa các label. Số liệu trong `ai-analysis-raw.md` là **số tính thật**, không bịa — cuộc rà soát dưới đây đối chiếu số đó với `analyze_jtl.py` chạy trên đúng 4 file đã chấm điểm.

1. Phân tích do AI thực hiện

Prompt và output nguyên văn ở `reports/ai-audit-report.md` Entry #1; mục này tóm tắt kết luận và ngưỡng mà AI đề xuất, trích nguyên văn.

#	Kết luận / ngưỡng AI đưa ra	Entry
1	"All four runs pass a standard 1% error-rate SLA... Stress and Load are perfectly clean at 0.00%."	#1
2	"983.78 req/s sustained with 0.00% errors and p95 = 4ms is a strong result — this backend can clearly serve on the order of 1,000 requests/second in production without degrading."	#1
3	"I'd set the production capacity alarm around 800 req/s (80% of what was cleanly demonstrated here)."	#1
4	"Spike out-throughputs Stress by ~59%... the 'Stress' scenario in this suite is the more conservative of the two and Spike is not actually the riskiest test."	#1
5	"checkout latency at p95 is only 139ms under spike conditions — well inside the 200ms threshold that's typical for a checkout endpoint."	#1
6	"No sign of a memory leak or slow degradation from this file alone (a proper leak check would need a memory/RSS time series alongside this, which isn't in the <code>.jtl</code>)."	#1
7	Ngưỡng đề xuất: error rate fail > 1%; p95 all-endpoints fail > 200ms; p99 fail > 500ms; throughput floor fail < 700 req/s.	#1
8	5 đề xuất tối ưu hoá: index catalog, connection pool, cache <code>GET /api/products</code> , rate-limit <code>reset-password</code> , horizontal scaling.	#1

2. Lỗi diễn giải — giá trị đúng trích từ log thô

#	AI nói	Giá trị đúng	Nguồn (.jtl + cách tính)	Bản chất lỗi
1	Kết luận #1: Spike "passed cleanly" ở 0.03% lỗi gộp, dưới ngưỡng 1%	0.03% là con số gộp trên cả 7 label. Toàn bộ 157 lỗi nằm ở một label duy nhất — 02 POST /api/reset-password — với error % thật của label đó là 0.235% (157/66.764), còn 6 label kia là 0.00%. Và trong chính label đó, lỗi chỉ xảy ra trong hai cửa sổ đột biến (t=90–150s: 81 lỗi; t=210–270s: 76 lỗi), 0 lỗi ngoài hai cửa sổ.	gunzip -c perf/results/jtl/23127300_Spike_20260814.jtl.gz > /tmp/x.jtl && python3 perf/scripts/analyze_jtl.py /tmp/x.jtl → dòng 02 POST /api/reset-password 66764 157 0.23 ... ; cửa sổ thời gian đối chiếu ở reports/run-manifest.md mục "Lỗi: lỗi thật của SUT so với nhiều do bộ khung kiểm thử".	Error % gộp che mất việc lỗi tập trung ở đúng một label và đúng hai cửa sổ tải — đây chính là dạng lỗi task-22-brief.md liệt kê: "an aggregate error % quoted while the errors cluster in a single label".
2	Kết luận #5: "checkout latency at p95 is only 139ms under spike conditions"	139ms là p95 của dòng ALL — gộp cả 7 label (kể cả 01 POST /api/forgot-password với p95=152ms và 03 POST /api/login với p95=97ms). p95 riêng của 07 POST /api/checkout là 82ms — thấp hơn số AI trích gần 70%.	Cùng lệnh trên: dòng 07 POST /api/checkout ... p95 82 so với dòng ALL ... p95 139 .	AI đọc percentile từ dòng tổng hợp rồi gán cho một label cụ thể (checkout) mà không kiểm tra dòng per-label tương ứng — đúng dạng "percentiles quoted from the wrong label".
3	Kết luận #2, #3, #4: "this backend can clearly serve ~1,000 req/s in production", đặt alarm ở 800 req/s, và so throughput Spike vs Stress để kết luận Spike "không phải bài test rủi ro nhất"	Throughput trong cả hai file phản ánh hình dạng arrival của workload model (Stress dùng think-time đã siết theo khuyến nghị calibration; Spike có hai cửa sổ đột biến bỏ hẳn think-timer), không phải một trần năng lực đã đo được. CPU đỉnh của SUT trong Stress chỉ 61.2% của 1 lỗi, trong Spike là 126.7% của 1 lỗi — trên máy 12 lỗi, tức chưa tới 11% tổng năng lực CPU của máy ngay cả ở bài test nặng nhất. Bộ calibration quét riêng cả trục concurrency (25–300 luồng) lẫn trục arrival-rate (xuống tới zero think time) và không tìm được điểm gãy nào do SUT gây ra.	reports/run-manifest.md cột "CPU đỉnh của SUT" (Stress 61.2%, Spike 126.7%); perf/results/calibration/calibration-20260814.md mục "Final conclusions" — "Stress ceiling: not found; no evidence supports any specific number."	File .jtl không mang tín hiệu tài nguyên (CPU/RSS) và không mang mô hình think-time/burst của kịch bản, nên throughput thô bị đọc thành năng lực máy chủ đã kiểm chứng — đúng dạng "throughput read as server capacity when it includes think time", ở đây là bị đọc mà không biết trục arrival-rate đã bị đổi giữa hai kịch bản.

#	AI nói	Giá trị đúng	Nguồn (.jtl + cách tính)	Bản chất lỗi
4	Đề xuất tối ưu #4: rate-limit POST /api/reset-password vì đây là "the only endpoint that produced any errors" — ngụ ý cần phòng lạm dụng	157 lỗi HTTP 400 này là hiều do bộ khung kiểm thử , không phải hành vi SUT: CSV Data Set Config dùng một con trỏ dùng chung cho 620 luồng trong Spike, và trong hai cửa sổ đột biến, hai luồng có thể cùng giữ một hàng tài khoản — luồng sau ghi đè resetToken của luồng trước trước khi luồng trước kịp dùng. Không lần chạy Load/Stress/Endurance nào (cùng label, cùng SUT) có bất kỳ lỗi nào ở label này.	reports/run-manifest.md mục "Lỗi: lỗi thật của SUT so với nhiều do bộ khung kiểm thử", dòng Spike (157 tổng, 0 lỗi thật của SUT, 157 nhiều do bộ khung); tái lập bằng lệnh ở dòng 1 của bảng này.	Gán một artefact của dữ liệu kiểm thử (con trỏ CSV dùng chung) cho một đặc tính cần vớ ở tầng sản phẩm — attribute sai nguồn gốc lỗi, không phải lỗi tính toán số học.

3. Đánh giá các đề xuất tối ưu hoá của AI

#	Đề xuất	Feasible / Hallucinated	Lý do
1	Thêm index cho các trường dùng ở endpoint danh sách/chi tiết sản phẩm	Hallucinated (vô nghĩa với dữ liệu này)	perf/data/products.csv chỉ có 5 dòng . Index trên một bảng 5 dòng không tạo khác biệt đo được — cả 04 GET /api/products lẫn 05 GET /api/products/{id} đã trả lời ở mean 1.4–1.9ms tại mọi mức tải đã chạy, kể cả throughput gộp ~1565 req/s của Spike.
2	Thêm connection pool cho tầng database	Hallucinated (không khớp kiến trúc)	Backend giữ một kết nối database dùng chung cho cả tiến trình (đã xác lập trong quá trình làm việc trước đó của dự án này), không phải một client đa kết nối mà connection pool nhằm tới giải quyết. Điểm nghẽn ghi thật sự (nếu có) là tuần tự hoá ghi trên một file dữ liệu dùng chung — pooling kết nối ở tầng ứng dụng không đổi được điều đó.
3	Cache response GET /api/products với TTL 30–60s	Feasible về mặt kỹ thuật, nhưng không có bằng chứng đo được nào biện minh	04 GET /api/products đã ở mean 1.4–1.9ms tại mọi mức tải trong cả 4 lần chạy; cache một bảng 5 dòng không có chỗ nào để cải thiện đo được so với baseline này.
4	Rate-limit POST /api/reset-password vì 157 lỗi	Hallucinated (chẩn đoán sai nguồn gốc)	Như mục 2.4 ở trên: 157 lỗi là artefact của con trỏ CSV dùng chung trong bộ khung kiểm thử, không phải traffic lạm dụng. Rate-limit một luồng khôi phục mật khẩu hợp lệ vì một lỗi của công cụ đo sẽ ảnh hưởng người dùng thật mà không sửa được nguyên nhân thật.
5	Horizontal scaling / load balancer vì Spike đạt >1500 req/s	Hallucinated	EShop lưu dữ liệu trên một file SQLite duy nhất (đã xác lập trong quy tắc làm việc của dự án này) — thêm instance mà không giải quyết tầng lưu trữ dùng chung không tăng năng lực, mà tạo rủi ro tranh chấp ghi mới. Cũng không có bằng chứng nào trong 4 lần chạy hay trong calibration cho thấy một trần năng lực đã bị chạm tới mà scaling cần trả lời.

4. Nhận xét chung

File .jtl không mang theo ngữ nghĩa lược đồ: nó không nói label nào là bước cuối cùng, quan trọng nhất của một hành trình (checkout), không nói think-time hay burst design khác nhau giữa hai kịch bản, và không mang theo bất kỳ tín hiệu tài nguyên nào (CPU/RSS) để phân biệt "hệ thống còn thừa sức" với "hệ thống đang ở giới hạn". Bốn lỗi diễn giải ở mục 2 đều rơi đúng vào khoảng trống đó: gộp lỗi qua toàn bộ

label (mục 2.1), gán percentile của dòng tổng hợp cho một label cụ thể (mục 2.2), đọc throughput thô thành năng lực đã kiểm chứng dù không có tín hiệu CPU/RSS đi kèm (mục 2.3), và gán một artefact của công cụ đo cho một đặc tính cần vá ở sản phẩm (mục 2.4).

Điều đáng chú ý là phần **arithmetic thô của AI đúng** — 0.03% lỗi gộp, p95 gộp 139ms, throughput 983.78/1565.53 req/s đều khớp với `perf/results/ground-truth.txt` (chạy `analyze_jtl.py` trên đúng 4 file đã chấm điểm) trong sai số làm tròn. Lỗi không nằm ở phép tính mà ở tầng diễn giải phía trên phép tính — đúng là lớp lỗi mà một mô hình không có ngữ cảnh domain (workflow là gì, kịch bản nào siết think-time thế nào, tài nguyên máy đang ở đâu) không có cách nào tránh được chỉ bằng cách đọc kỹ hơn cùng một file. Ba trong bốn đề xuất tối ưu hoá "hallucinated" ở mục 3 cũng cùng gốc: chúng là những khuyến nghị hợp lý *cho một kiến trúc chung chung*, nhưng không khớp với kiến trúc cụ thể (catalog 5 dòng, một kết nối DB, một file SQLite) mà `.jtl` không có cách nào tiết lộ.

Một điểm AI làm đúng đáng ghi nhận: ở kết luận #6 (Endurance), AI tự nhận "a proper leak check would need a memory/RSS time series... which isn't in the `.jtl`" thay vì đoán bừa — đúng giới hạn thật của dữ liệu nó có. Tương phản với bốn lỗi ở mục 2, đây là bằng chứng lỗi diễn giải ở đây không phải do model "lười", mà do model tự tin diễn giải vượt quá dữ liệu nó thực sự có khi câu hỏi mời gọi một câu trả lời cụ thể (ngưỡng, kết luận về capacity) nhưng lại im lặng đúng lúc khi câu hỏi không đủ dữ liệu để trả lời chắc chắn.